

Practical No. 10

Aim: To configure load balancing and auto scaling in AWS to distribute incoming traffic across multiple EC2 instances and automatically adjust resources based on demand.

Theory:

Elastic Load Balancing (ELB) automatically distributes incoming traffic across multiple EC2 instances in one or more Availability Zones, improving fault tolerance. The Application Load Balancer (ALB) operates at Layer 7 and routes traffic based on the content of the request.

Amazon EC2 Auto Scaling automatically adjusts the number of running instances based on CloudWatch metrics such as CPU utilization, scaling out when demand increases and scaling in when it decreases.

Key Concepts

- Application Load Balancer — Distributes HTTP/HTTPS traffic across multiple instances
- Target Group — Defines the set of instances the load balancer routes traffic to
- Launch Template — Defines the configuration used to launch new EC2 instances in an Auto Scaling group
- Auto Scaling Group — Manages the number of running instances based on scaling policies
- CloudWatch Alarms — Monitor metrics and trigger scaling actions when thresholds are crossed

Advantages

- High availability by distributing traffic across multiple Availability Zones
- Cost efficiency by automatically scaling down instances during low demand
- Fault tolerance — unhealthy instances are automatically replaced

Steps:

Task 1: Create an AMI for Auto Scaling

1. In EC2, select Web Server 1. From the Actions menu, choose Image and templates > Create image. Set Image name to WebServerAMI and click Create image.

Task 2: Create a Load Balancer

1. In Target Groups, click Create target group. Set type to Instances, name to LabGroup, VPC to Lab VPC, and click Create target group.
2. In Load Balancers, create an Application Load Balancer named LabELB. Select Lab VPC with Public Subnet 1 and 2, set security group to Web Security Group, forward listener to LabGroup, and click Create load balancer.

Task 3: Create a Launch Template and Auto Scaling Group

1. In Launch Templates, create a template named LabConfig with AMI: WebServerAMI, instance type: t2.micro, key pair: vockey, security group: Web Security Group, and Detailed CloudWatch monitoring enabled.
2. From LabConfig's Actions menu, create an Auto Scaling group named Lab Auto Scaling Group. Set VPC to Lab VPC and subnets to Private Subnet 1 and 2. Attach to LabGroup and enable CloudWatch group metrics.
3. Set Desired: 2, Min: 2, Max: 6. Add a target tracking policy named LabScalingPolicy with Average CPU Utilization at 60. Add tag Name: Lab Instance and click Create Auto Scaling group.

Task 4: Verify that Load Balancing is Working

1. In EC2 Instances, confirm two instances named Lab Instance are running. In Target Groups, select LabGroup and wait until both instances show healthy status.
2. Copy the DNS name of LabELB from Load Balancers and open it in a browser to confirm the web application loads successfully.

Task 5: Test Auto Scaling

1. Navigate to CloudWatch > All alarms. Confirm two alarms (AlarmHigh and AlarmLow) are visible.
2. In the web application, click Load Test to generate high CPU load. Return to CloudWatch and wait for AlarmHigh to enter In alarm state. In EC2 Instances, confirm additional Lab Instance instances have been launched automatically.

Task 6: Terminate Web Server 1

1. In EC2 Instances, select Web Server 1. From the Instance state menu, choose Terminate Instance and confirm. This instance is no longer needed as the Auto Scaling group manages new instances using the AMI.

Conclusion: Thus, we have successfully configured load balancing and auto scaling in AWS to distribute incoming traffic across multiple EC2 instances and automatically adjust resources based on demand.